Worksheet — 2.3 Algorithms

Name: __ **Date:** ____

OCR A Level Computer Science H446, Component 02, Section 2.3.1 — Algorithms.

Time: ~40 min. SHOW YOUR TRACES.

Activity 1 — Big O match

Match each algorithm to its **best**, **average** and **worst** time complexity by writing the correct Big O in each cell. Choose from: `O(1)`, `O(log n)`, `O(n)`, `O(n log n)`, `O(n²)`.

Algorithm	Best	Average	Worst
Linear search			
Binary search			
Bubble sort (with swap flag)			
Insertion sort			
Merge sort			
Quick sort			

Activity 2 — Trace a sort (bubble OR insertion)

Sort this list into **ascending** order: `[5, 1, 4, 2, 8]`

Circle which you are tracing: **BUBBLE / INSERTION**

Fill in the list state **after each pass**. Add rows if you need them.

Pass	List state after this pass	Swaps this pass
Start	5 1 4 2 8	—
1	__ __ __ __ __	
2	__ __ __ __ __	
3	__ __ __ __ __	
4	__ __ __ __ __	

Final sorted list: `__ __ __ __ __`

Activity 3 — Trace a binary search

Sorted list (index 0–8):

Index	0	1	2	3	4	5	6	7	8
Value	3	7	11	15	19	23	27	31	35

Search for 27. Use `mid = (low + high) DIV 2` (integer division, round down). Fill the table.

Step	low	high	mid (index)	value at mid	< = > target?
1					
2					
3					
4					

Found at index: __ **Number of comparisons:** ____

Activity 4 — Big O justification (short answer)

(a) Explain why binary search is **O(log n)**. Refer to what happens to the list each comparison.

_____

_____

_____

(b) Explain why bubble sort is **O(n²)** in the average case.

_____

_____

(c) State the precondition binary search requires that linear search does not.

_____

Activity 5 — Tree traversals

Binary tree (text form):

Write the output of each traversal:

Pre-order (Node → Left → Right):

`____ ____ ____ ____ ____ ____ ____`

In-order (Left → Node → Right):

`____ ____ ____ ____ ____ ____ ____`

Post-order (Left → Right → Node):

`____ ____ ____ ____ ____ ____ ____`

Which traversal outputs a BST in ascending order? __

Activity 6 — Dijkstra's shortest path

Weighted, undirected graph given as an **edge list** (node1 — node2 : weight):

```
A — B : 4
A — C : 2
B — C : 1
B — D : 5
C — D : 8
C — E : 10
D — E : 2
D — F : 6
E — F : 3
```

Find the shortest path and total distance from A to F.

Working table — write the tentative distance to each node, updating as you settle nodes. Cross out / overwrite when a shorter route is found.

Settled node (order)	**A**	**B**	**C**	**D**	**E**	**F**
Initial	0	∞	∞	∞	∞	∞

Shortest path A → F: `____ → ____ → ____ → ____ → ____`

Total distance: _____

Challenge

Challenge box. A* search uses `f(n) = g(n) + h(n)`.

(a) For the graph above, suppose you are given a straight-line heuristic estimate to F of: `h(A)=9, h(B)=7, h(C)=8, h(D)=4, h(E)=3, h(F)=0`. Is this heuristic **admissible**? Explain how you would check, and what admissible means.

(b) State ONE situation where A *explores fewer nodes than Dijkstra, and ONE condition under which A* behaves exactly like Dijkstra.

```
```

Answer key

Activity 1 — Big O match

Algorithm	Best	Average	Worst
Linear search	O(1)	O(n)	O(n)
Binary search	O(1)	O(log n)	O(log n)
Bubble sort (with swap flag)	O(n)	$O(n^2)$	$O(n^2)$
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$
Merge sort	O(n log n)	O(n log n)	O(n log n)
Quick sort	O(n log n)	O(n log n)	$O(n^2)$

Note: bubble sort best case is O(n) **only with a swap flag** on already-sorted data. Quick sort worst case is $O(n^2)$ with a poor pivot (e.g. already-sorted data, first/last element pivot).

Activity 2 — Sort trace

BUBBLE SORT of `[5, 1, 4, 2, 8]` (compare adjacent pairs, swap if left > right; largest bubbles to the end each pass):

Pass	List state after this pass	Swaps this pass
Start	5 1 4 2 8	—
1	1 4 2 5 8	5↔1, 5↔4, 5↔2 = 3 swaps
2	1 2 4 5 8	4↔2 = 1 swap
3	1 2 4 5 8	0 swaps → with swap flag, algorithm stops here

Final sorted list: `1 2 4 5 8`

Pass-by-pass detail of pass 1 (each adjacent comparison): - (5,1)→swap → 1 5 4 2 8 - (5,4)→swap → 1 4 5 2 8 - (5,2)→swap → 1 4 2 5 8 - (5,8)→no swap → 1 4 2 5 8

INSERTION SORT alternative (acceptable if student chose insertion) — list after each pass (element inserted into sorted front):

Pass	List state after this pass
Start	5 1 4 2 8
1 (insert 1)	1 5 4 2 8
2 (insert 4)	1 4 5 2 8
3 (insert 2)	1 2 4 5 8
4 (insert 8)	1 2 4 5 8

Final sorted list: `1 2 4 5 8` (same result).

Activity 3 — Binary search for 27

List: indices 0–8, values 3, 7, 11, 15, 19, 23, 27, 31, 35.

Step	low	high	mid (index)	value at mid	< = > target?
1	0	8	4	19	19 < 27 → search right, low = mid+1 = 5
2	5	8	6	27	27 = 27 → FOUND

Found at index: **6**. Number of comparisons: **2**.

(Working: step 1 mid = (0+8) DIV 2 = 4; step 2 mid = (5+8) DIV 2 = 6.)

Activity 4 — Justifications

(a) Binary search is O(log n) because **each comparison halves the size of the list still to be searched** — it discards the half that cannot contain the target. The number of times n can be halved before reaching 1 is $\log_2(n)$, so the number of comparisons grows logarithmically with n. (Doubling the list size adds only one extra comparison.)

(b) Bubble sort is $O(n^2)$ because it uses **two nested loops**: an outer loop of up to n passes, and for each pass an inner loop making up to n adjacent comparisons. That gives roughly $n \times n = n^2$ comparisons, so time grows with the square of the input size.

(c) Binary search requires the list to be **sorted**; linear search works on unsorted data.

Activity 5 — Tree traversals

Tree: root 40; left subtree 20 (children 10, 30); right subtree 60 (children 50, 70).

- **Pre-order** (Node, Left, Right): `40, 20, 10, 30, 60, 50, 70`
- **In-order** (Left, Node, Right): `10, 20, 30, 40, 50, 60, 70`
- **Post-order** (Left, Right, Node): `10, 30, 20, 50, 70, 60, 40`

Ascending order is produced by the **in-order** traversal of a BST.

Activity 6 — Dijkstra A to F

Edges (undirected): A-B 4, A-C 2, B-C 1, B-D 5, C-D 8, C-E 10, D-E 2, D-F 6, E-F 3.

Settle the unvisited node with the smallest tentative distance each round; relax its neighbours.

Settled (order)	A	B	C	D	E	F
Initial	0	∞	∞	∞	∞	∞
Settle A (0)	0	4	2	∞	∞	∞
Settle C (2)	0	3	2	10	12	∞
Settle B (3)	0	3	2	8	12	∞
Settle D (8)	0	3	2	8	10	14
Settle E (10)	0	3	2	8	10	13
Settle F (13)	0	3	2	8	10	13

Relaxation notes: - From A: B = 4, C = 2. - Settle C (2): B = min(4, 2+1) = 3; D = 2+8 = 10; E = 2+10 = 12. - Settle B (3): D = min(10, 3+5) = 8; (C already settled). - Settle D (8): E = min(12, 8+2) = 10; F = 8+6 = 14. - Settle E (10): F = min(14, 10+3) = 13. - Settle F (13): done.

Shortest path A → F: `A → C → B → D → E → F`

Total distance: 13.

(Check: A→C 2, C→B 1, B→D 5, D→E 2, E→F 3 = 2+1+5+2+3 = 13. Compare alternative A→C→B→D→F = 2+1+5+6 = 14, longer. So 13 is optimal.)

Challenge

(a) A heuristic is **admissible** if it **never overestimates** the true shortest-distance from a node to the goal F. To check, compare each h(n) to the actual shortest distance to F (computed by Dijkstra from each node): - True shortest distances to F — F:0, E:3, D:5 (D→E→F = 2+3), B:10 (B→D→E→F = 5+2+3), C:11 (C→B→D→E→F = 1+5+2+3), A:13. - Heuristic given: A=9, B=7, C=8, D=4, E=3, F=0. Each is ≤ the true distance (9≤13, 7≤10, 8≤11, 4≤5, 3≤3, 0≤0), so the heuristic **is admissible** and A* would find the optimal path.

(b) A *explores fewer nodes than Dijkstra when a good (informative) heuristic directs the search toward the goal instead of expanding outward in all directions — useful when you only need the path to one goal node. A* behaves **exactly like Dijkstra** when the heuristic is **h(n) = 0** for every node (no information), because then f(n) = g(n).